Introduction

The Project i have created and developed for the Generative Art module was made to explore an interactive and user friendly simulation that visually reconstructs images by drawing thousands of randomised or grid aligned lines that are each colour coordinated to match the pixel data that is sourced from an image. The aesthetic outcome that i produce is intended to be abstract however also recognisable depending on the way that the user uses my project to reinterpret the images given through chance and control over the simulation. The concept I used behind this project is in the transformation of visual media using an algorithmic process in coding and in particular using p5.js and its resources. Rather than manipulating images in conventional mundane ways I had the goal to set out and generate new interpretations of images using code by overall breaking down the images visual data into pixel based colours, then reconstructing them altogether through directional lines that reflect the specific colours of pixels in the chosen image. To enhance the creative interaction and the users control over my simulation I implemented a user interface that includes sliders and toggle buttons which allows real time changes to parameters in this project such as the length of each line, the grid scale, the speed of the lines produced and the image selection.

This documentation will outline the conceptual motivations behind the project and its overall core goals and aims as well as the struggles and errors i came against and how they were solved and the functions used to create these goals along with an evaluation of the final result and what could have gone better/worse. In this write up I will explain every function I have used and the structural element of the code that I will examine in depth demonstrating the technical understandings of this generative art project. In this project I aim to generate images with the ability to give users a way to intervene in the generative art process and allow them to make and interpret art of their own.

Project Goals and Concept

The goal of this project that I set out to complete was to develop a generative art work that will take digital images and then reconstruct these images visually using a line based abstraction to create an aesthetic art piece by the user. The project i created focuses on interpreting images data, i aimed to specifically focus on the pixel level colour information and brightness mapping of images transforming this into a dynamic visual made entirely of just lines that progress to create a piece of art by the user. The lines in my project are not randomly placed without context, instead each is drawn using the colour of each pixel that is being extracted from the underlying image, making the final outcome an overall reinterpretation of the original image however using the generative principles in which i implemented.

One of the key conceptual objectives that i set out to create when making this project was to allow a balance between control and chaos. I believe that this is a frequent theme that is involved in generative art, as while the algorithm that i create does follow the rules of the code i apply it does also embrace randomness in areas such as direction, position and colour blending which can easily get out of hand if the right procedures and implementation

is not applied. My program brings together this tension by allowing my users to switch between the randomised line placement and the structured grid based generation giving the user more control over the effects that can be outputted. The result from this project is a tool that can be used to produce both a chaotic scene of coloured vibrant line and imagery and a more ordered visually coherent outcome of coloured lines and grid pixel image art depending on what the users choices are.

Another element in the project I intended on making clear was that the project is very much interactive instead of being static. This interactivity that I include in this simulation was realised through the inclusion of user interface elements such as the sliders that I added and the toggle buttons. These controls allow the viewer and artists of my code to engage with the generative process and then tweak the overall visual outcome in real time. For instance the sliders that i created allow for my users to change the length of the lines that are produces form the image which changes the art significantly from small lines having a more detailed but still translucent effect to having larger lines that make the original image harder to interpret but having the outcome of a very translucent image with cool coloured visual effects. The grid scale allows the user to create circle block like effects which are distorted from the original image and the grid scale slider allows this to enhance or decrease the scale of the grid that is shown when the user interacts with the image. The number of lines drawn does simply draw the lines that are created much quicker to produce the output at a faster rate, however when the lines are drawn slower while the art is being drawn the user can use this to still make different effects on the image. This approach that I have created lines in with the understanding of procedural generative art where in that the tool is as much part of the creative process as the final image is.

Basic Goal and How I Proposed to Accomplish It

My basic goal in this project was to create a generative system that in turn visually deconstructs and also reconstructs images using lines that produce overtime onto a canvas reflecting their original colours and pixel locations. To accomplish this I first needed to build a program in which I could load images and access their individual pixel data. From there I needed a rendering system that would allow me to take random or structured (grid based) coordinates from the image and then draw lines outward from those points using the corresponding colour of the pixel underneath.

The initial concept that I intended to create stemmed from a classroom exercise on brightness mapping where I learned how pixel brightness could be used to control the size, shape or density of visual elements that are drawn over an image. In the earlier versions of my project I began to experiment with mapping the brightness values to alter circle sizes or to change the transparency of drawn shapes. Although these experiments were successful in understanding how to manipulate image pixels I found the visual results less appealing than what I was hoping to accomplish with my project.

Overall this led me to think more outside the box, rather than just working with brightness and a grayscale abstraction of the image. I wanted to access the full RGB data from each pixel and then use this data to generate vibrant colours and dynamic visuals. From this idea

the final project began to take shape with a system that re-defines an image by drawing lines based on each pixel's true colour and not just as its brightness values and then positioning those lines in a randomised or grid structured way.

To accomplish this I needed to build myself and code a program in which can load multiple images, access their personalised pixel data and then apply a custom drawing routine. The key visual technique I chose was line drawing where each line's starting point would correspond to a pixel's location and its colour directly reflecting the underlying pixel and its attributes.The direction of the line could be randomised for expressive variation or (in the case of the idea that i came up with later along in the project) controlled by a grid system to produce a more structured art piece if that is what the user desires. The transition i went through from brightness based mapping to a full colour image deconstruction i think marked a turning point in both the visual quality, conceptual depth and technical quality of my project pushing myself to go further.

To help with this project I proposed to use p5.js with their creative coding library to implement this idea as its built-in tools for loading images, reading pixel arrays and drawing shapes made this well suited for this type of generative task with my project. With `createSlider` and `createButton` I realised I could implement real time interactivity which would give my users full control over the core parameters such as the line length, grid scale and the number of lines that are drawn. This approach with my project would not only allow the user to observe generative art but to participate in shaping it and making it their own.

Functions and My Process

In this section I will explain the details of the core functions used in the development of my generative art project and I will explain their individual roles and how they contribute to the system overall. It demonstrates the technical work behind the canvas, the design thinking and the structural decisions that shape the final implementation of this project overall. The resource that I used to help with the learning and discovery of these functions was the p5.js reference tools, this helped to make sure i was implementing the sliders and everything else correctly.

Implementing the setup() Function and Initializing the canvas and User Interface

The `setup()` function serves as the initialization point for the entire sketch, and it runs once when the program starts. In my project the `setup()` first creates a canvas which i used the dimensions of 5,000 by 5,000 pixels to ensure that the images lines are not being restricted and can be drawn outside of the images area. The code line `img = images[currentImageIndex];` loads the image at the starting index of the preloaded image array which overall is very essential to my project because the entire of the generative drawing logic operates based on the pixels from this image.

Along with this the function implements interactive user interface elements. A button that I created is used to allow users to change the current image that is being processed. This button is styled with some rounding edges and the largest possible readable font to attempt to make the interaction visually clear. Next I initialised three sliders into my project and positioned them on the canvas. I created one for adjusting the line length `lineLengthSlider`, one for adjusting the grid scale `gridSizeSlider` and then one for determining the number of lines that are produced over time (faster or slower lines produced) `numOfLinesSlider`. Each of the sliders are styled for visibility and then descriptive labels are added with large shadowed text to make sure that the letters stand out against different and all coloured backgrounds of my images. The Grid scale slider in particular allows the user to control whether the image is reconstructed with evenly spaced lines or through a randomized placement that has the outcome effect of small circles that draw to the screen filled with lines and colours corresponding to the image. A separate toggle button was added and also linked to the `toggleGridScale` function and when clicked it visually changes colour to green based on whether the user has selected this type of drawing mode in my program, this helios users that are viewing my project understand whether the grid based drawing is active or not.

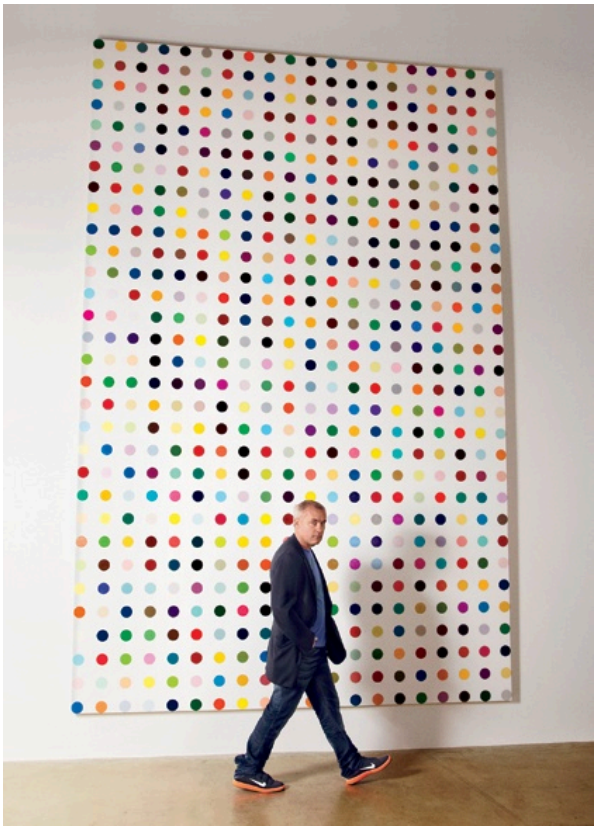
Explaining the draw() function and Pixel mapping / Line Rendering

The `draw()` function in my program is operating continuously in the background to reinterpret the pixel data of the loaded image in a visually dynamic way. At the beginning of each frame of my images `img.loadPixels();` is called in this to load the images raw pixel data into the pixels array this enables the access to the RGB values of each pixel.

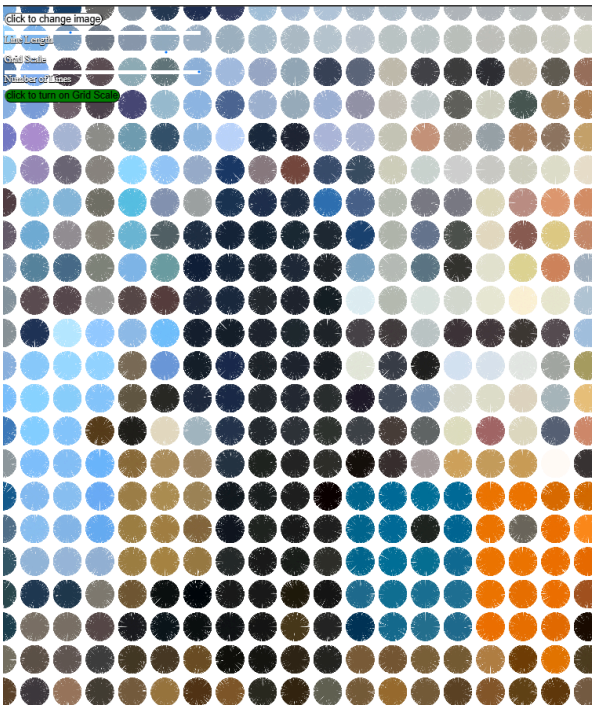
The drawing process then begins to go into another direction based on the boolean variable that i added being the `useGridScale` function. If `useGridScale` is true in this program then the system enters the grid based drawing mode. Within this mode the canvas is looped through systematically with step sizes based on the users selected `gridScale` value. At each grid point my code then computed all of the pixels data index into the array and then retrieves their RGB colour values. These values that are retrieved are then transformed into a colour object using the code `color(r, g, b, a);`, this colour is then applied as the stroke for one line. After this a random angle is made using `random(TWO_PI);` and a line is drawn from the grid point outward by a distance which is equal to the user selected `lineLength` using the basic trigonometric functions (cos and sin) which are used to determine the end point coordinates.

If the `useGridScale` boolean is false during the drawing process instead it uses random coordinates for the starting point of each line. For each
Of the user selected number of lines (`numOfLines`) a random x and y coordinate location is chosen on the image. The exact same colour extraction and the angle generation also applies here however though because both of the position and orientation of the lines are

random then the final visual result is more chaotic and abstract. This flexibility allows for an expressive range of outcomes from the same image source. One thing i would like to note is that when the grid based mode is active it often holds the results the visually resemble Damien Hirst's structured dot paintings which are however formed through the directional lines rather than circle spots through due to the evenly placed grid scale and the strong colour effects that this generates



Damien Hirst



Generative Art project

This realisation of the two shows that the user really can create some very interesting artwork using the sliders and gridscale toggle.

The Change Image Function Cycling Through Images In An Array

The `changeImage` function allows the users to switch between different visual inputs. Each time that the image `currentImageIndex` by one, using a loop back to the start of the image array when the end is reached. The updated image is then assigned to `img` and the `resizeCanvas(` function is called to adapt the size of the canvas to the new images dimensions. This very smooth transition between images then ensures that the drawing logic can be applied dynamically to a wide variety of visual styles then encouraging experimentation and exploration within the generative system.

The toggleGridScale() Function And Switching Between Drawing Modes

The `toggleGridScale()` function is critical in my project for the user interaction side of this artwork and the dynamic control over visual style. The function operates by toggling the boolean variable `useGridScale` (as spoke about before) using the line `useGridScale = !useGridScale;`. This boolean variable that I added overall inverts the variable's current state meaning that if the grid scale was previously active then it becomes inactive and then vice versa. Depending on the new state weather it is active or not the function then will also change the background colour of the toggle button using some CSS styling meaning when the grid scale is active, the button turns green and when it is inactive it reverts back to white so the user understands when the toggle is active and not. The purpose of this function is to give users real time control over how the artwork is generated. The structured grid based method that i added produces an ordered representation of the image shown that corresponds oppositely to the chaotic lines that are being drawn when the mode is turned off. By allowing an easy toggle between these two different modes i think that this project offers multiple different styling outputs from the same project which i also think encourages my users to engage more with the generative techniques that i added.

The Sprite Class and Additional Elements

In addition to the main generative image rendering i added a custom `JS Sprite.js` class to develop and experiment with the dynamic and moving elements that could be layered over the generative drawings. While this isn't really essential to the primary functions of my project and the transforming of pixel data into lines, this class that I created demonstrates OOP (object oriented programming) in p5.js.

Defining The Sprite Class

The sprite class that i constructed uses the parameters `(x, y, w, h, c)` which corresponds to the sprites position, width, height and the colour. Each sprite also receives a random directional vector using the code `this.dirX = random(-1, 1);` for both the `dirX` and the `dirY` values which allow for movement.

```
class Sprite {
    constructor(x, y, w, h, c) {
      this.x = x;
      this.y = y;
      this.dirX = random(-1, 1);
      this.dirY = random(-1, 1);
      this.w = w;
      this.h = h;
      this.c = c;
      this.move = false;
    }
    show() {
      fill(this.c);
      noStroke();
      ellipse(this.x, this.y, this.w, this.h);
    }
    update() {
      // Update the sprite
      if (this.move) {
        this.x += this.dirX;
        this.y += this.dirY;
      }
    }
  }
```

Purpose And Implementation

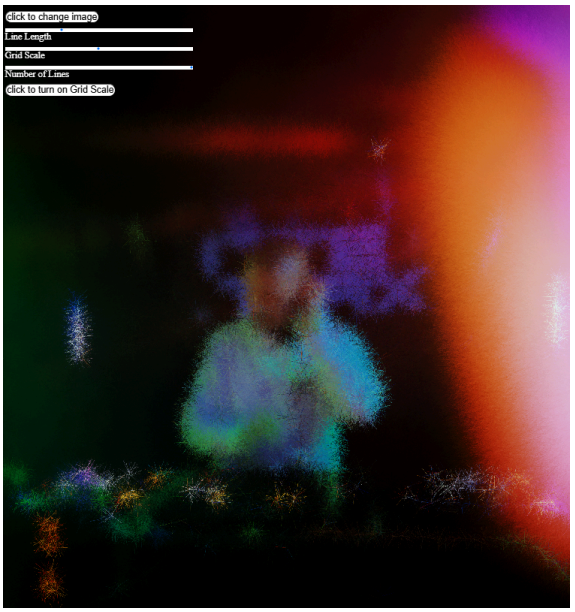
This class was initially created to explore how additional layers of generative art could be introduced uito the artwork. Although this is not actually integrated into the main drawing loop the sprites can easily be generated and displayed using the code `new Sprite(x, y, w, h, c)` and also using the show() and update() methods. When `move` is set to true then the sprites would animate themselves across the canvas responding to the directions.

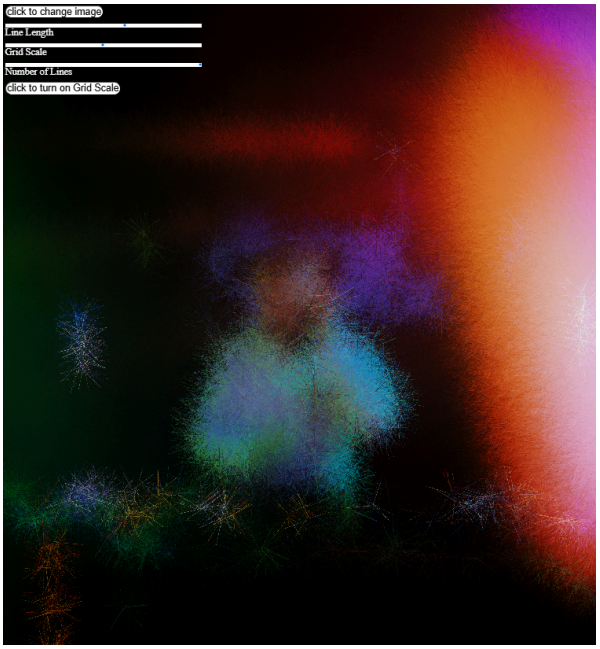
This idea was inspired through experiments in lessons where we used particles and shapes to respond to motion which osten simulated natural systems. Although this is not necessarily deeply embedded into the final artwork this class still serves as a sandbox for testing interaction and randomness in my generative design at the start of my project and shows the inner begging work that I came from to produce what I have done. In future iterations i think that this sprite class could be extended further for example it could be used to simulate the sprites that respond to the underlying image that is produced and its data it could move according to the brightness gradients or it could be used to trigger visual changes wherever they travel.

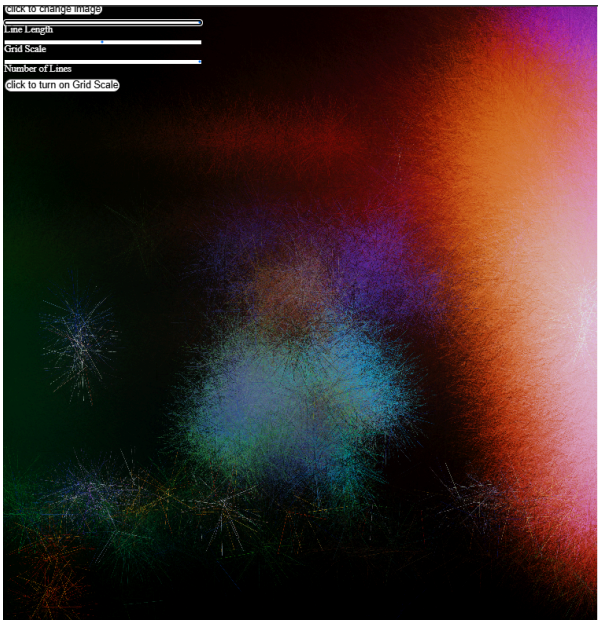
Produced Artwork

The outputs that are generated using this system can be highly diverse and very responsive to my users inputs. When the grid scale is active the output as spoken before looks a lot like a piece of Damien Hirst's iconic dot paintings. Although Hirst's work uses perfectly placed vivid coloured dots on a grid, the aesthetic created by my project also resembles the same effect when done correctly using the sliders as shown in the example i previously gave.

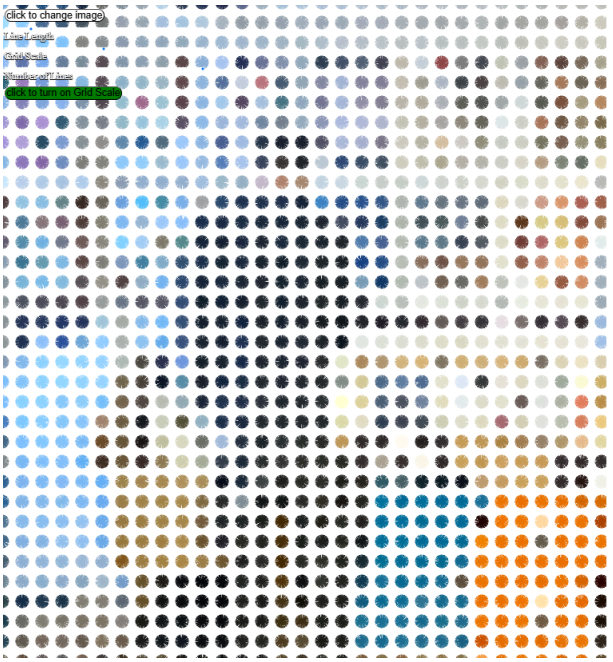
In contrast to this when the grid scale is turned off then the outputs can become a lot more chaotic and give a very translucent effect and a fragmented effect. This randomness that is shown is reflected in the scattering coloured lines that still maintain the images overall colour palate but slightly loose their formal structure depending on the values of the slider you use which results in an abstract art piece. The artwork that i will show below will show you the difference between the lines being smaller and larger showing the more detailed effect of smaller lines and then the more translucent effect of bigger lines. I found that these work well with colourful images.

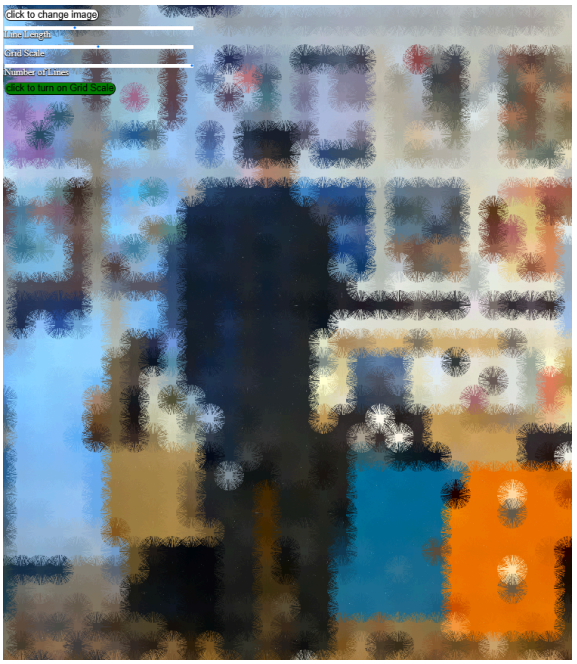






The next set of images that i will show are using the grid scale toggle to and also progressively upping the line length and also changing the grid scales size to show the more ordered and Damien Hirst like dot effects





Along with my project files i will also add a video of how this is done and show the sliders being used and the effects that they have and also the grid scale being used

Implementation Success and Overcoming Challenges

Initial Inspirations and Development Process

I started this project with a basic idea that we went over in class which was brightness mapping. I first used code that translated brightness levels in an image to change shape, color, or densities. It was through this experience that I learned the theory of how to use pixel data to create visual output. I wanted to take the idea a step further by not just taking brightness into account, but using the literal colour data of every pixel to draw lines onto the canvas. This change from simple brightness values to complete RGB colour extraction allowed the visual effect to become more vibrant and engaging.

The greatest strength of the implementation is the way it transforms ordinary photographs into artistic, painted equivalents with the help of unique techniques. The user interface (buttons, toggles, sliders) of the system makes the system interactive and simple to use, and it is capable of producing numerous creative outcomes with minimal effort. It is both a creative tool and an artwork generator.

Problems Faced During Development

Despite how the end project may look simple there was several setbacks i went through to get where i needed to be

Image loading and Scaling

Dealing with multiple image files was a bit tricky, especially when trying to keep the same quality on the canvas. Early versions of the program did not resize images correctly, causing images to be cropped or stretched. This was fixed by automatically resizing the canvas using `resizeCanvas(img.width, img.height);` after every new image is loaded. This will keep the artwork fitting the canvas, no matter what size the source image is.

```
let index = (floor(x) + floor(y) * img.width) * 4;
```

Pixel Indexing and the performance

Another technical issue was working with the pixels[] array efficiently. Drawing lines depending on pixel color involved correct indexing of the 1D array that holds 2D image data. Any one mistake could very easily result in reading out-of-bounds values or drawing errors. To solve this overall error the code i used to fix this was `let index = (floor(x) + floor(y) * img.width) * 4;`. This needed to be triple-checked and debugged to make sure that it never accessed invalid pixel data. Furthermore, rendering thousands of lines per frame (especially in random mode) might affect performance. To ensure this didn't happen, a limit was put on the number of lines through a slider `numOfLinesSlider`, allowing users to control the visuals.

```
button.style('background-color', 'white');
button.style('font-size', '50px');
button.style('border-radius', '50px');
```

The User Interface Clashing

Positioning UI controls (sliders, buttons, labels) in a way that does not obscure the artwork also required deliberate design. Controls in earlier versions overlapped or were positioned over generated art, creating visual noise. This was enhanced by visually defining control areas (at top/left) and the space to render the canvas. Decoration such as:

```
button.style('background-color', 'white');
button.style('font-size', '50px');
button.style('border-radius', '50px');
```

This made buttons and sliders easy to tell apart and look good together, helping create a friendly experience for users.

Evaluation

One of the most pleasing aspects of this project is the range of artistic outcomes it can produce. By loading various photos into the program and experimenting with the sliders, my users can then achieve a great variety of styles from soft, Translucent impressions to bold, messy abstract statements. When the grid scale toggle is on and the line placement is ordered, the resulting image looks interesting and almost like a pattern. This is because each square in the grid releases lines based on the colour of the pixels, similar to Damien Hirst's dot paintings. In his paintings, there is a tidy grid of colour, yet the colours also have some randomness to them. This similarity was not intended at first, but upon noticing it, it shows what good art can come out of my project